

CHAPTER 2 NODE JS





Chapter 2: Advanced Node.js Backend - Data Management & API Design

Great job completing Chapter 1! Now let's dive deeper into **advanced backend concepts** and build more complex features.



Chapter 2 Learning Goals

By the end of this chapter, you'll understand:

- **Advanced Database Operations** (queries, relationships, aggregation)
- **Data Validation & Sanitization**
- **File Upload & Management**
- **Email Integration**
- **Advanced Authentication** (roles, permissions, password reset)
- **API Documentation**
- **Performance Optimization**
- **Testing Your Backend**



Advanced Database Operations

1. Complex Queries

```
// auth.controller.js - Advanced user queries
```

[Copy](#)

```
// Get users with pagination and filtering
```

```
const getUsers = async (req, res) => {
```

```
  try {
```

```
    // Extract query parameters
```

```
    const page = parseInt(req.query.page) || 1;
```

```
    const limit = parseInt(req.query.limit) || 10;
```

```
    const search = req.query.search || '';
```

```
    const role = req.query.role || '';
```

```
    // Build filter object
```

```
    let filter = {};
```

```
    if (search) {
```

```
      filter.$or = [
```

```
        { name: { $regex: search, $options: 'i' } }, // Case insensitive
```

```
        search
```

```
        { email: { $regex: search, $options: 'i' } } ]
```

```
    ];
  }

  if (role) {
    filter.role = role;
  }

  // Calculate skip value for pagination
  const skip = (page - 1) * limit;

  // Execute query with pagination
  const users = await User.find(filter)
    .select('-password') // Exclude password field
    .sort({ createdAt: -1 }) // Sort by newest first
    .skip(skip)
    .limit(limit);

  // Get total count for pagination
  const total = await User.countDocuments(filter);

  res.status(200).json({
    success: true,
    data: users,
    pagination: {
      page,
      limit,
      total,
      pages: Math.ceil(total / limit)
    }
  });
} catch (error) {
  res.status(500).json({
    success: false,
    message: error.message
  });
}
};

// Get user statistics
const getUserStats = async (req, res) => {
  try {
    const stats = await User.aggregate([
      {
        $group: {
          _id: '$role',
          count: { $sum: 1 },
          avgAge: { $avg: '$age' }
        }
      }
    ]
  )
}
```

```
    },
    {
      $sort: { count: -1 }
    }
  ]);

const totalUsers = await User.countDocuments();
const activeUsers = await User.countDocuments({ isActive: true });

res.status(200).json({
  success: true,
  data: {
    totalUsers,
    activeUsers,
    roleDistribution: stats
  }
});

} catch (error) {
  res.status(500).json({
    success: false,
    message: error.message
  });
}
};
```

2. Database Relationships

Let's create a **Post** model that relates to Users:

```
// models/Post.model.js
const mongoose = require('mongoose');

const postSchema = new mongoose.Schema({
  title: {
    type: String,
    required: [true, 'Please add a title'],
    trim: true,
    maxlength: [100, 'Title cannot be more than 100 characters']
  },
  content: {
    type: String,
    required: [true, 'Please add content'],
    maxlength: [2000, 'Content cannot be more than 2000 characters']
  },
  author: {
    type: mongoose.Schema.ObjectId,    // Reference to User model
```

 Copy

```
    ref: 'User',                                // Which model to reference
    required: true
  },
  tags: [{
    type: String,
    lowercase: true
  }],
  likes: [{
    user: {
      type: mongoose.Schema.ObjectId,
      ref: 'User'
    },
    createdAt: {
      type: Date,
      default: Date.now
    }
  }],
  comments: [{
    user: {
      type: mongoose.Schema.ObjectId,
      ref: 'User',
      required: true
    },
    text: {
      type: String,
      required: true,
      maxLength: [500, 'Comment cannot be more than 500 characters']
    },
    createdAt: {
      type: Date,
      default: Date.now
    }
  }],
  status: {
    type: String,
    enum: ['draft', 'published', 'archived'],
    default: 'draft'
  },
  featuredImage: {
    type: String,
    default: null
  },
  viewCount: {
    type: Number,
    default: 0
  }
}, {
  timestamps: true,
  toJSON: { virtuals: true },
```

```
    toObject: { virtuals: true }
  });

// Virtual fields (computed fields)
postSchema.virtual('likesCount').get(function() {
  return this.likes.length;
});

postSchema.virtual('commentsCount').get(function() {
  return this.comments.length;
});

// Middleware to populate author on find
postSchema.pre(/^find/, function(next) {
  this.populate({
    path: 'author',
    select: 'name email avatar'
  });
  next();
});

module.exports = mongoose.model('Post', postSchema);
```

3. Post Controller with Relationships

```
// controllers/post.controller.js
const Post = require('../models/Post.model');
const User = require('../models/User.model');

// Create a new post
const createPost = async (req, res) => {
  try {
    // Add user to request body
    req.body.author = req.user.id;

    const post = await Post.create(req.body);

    res.status(201).json({
      success: true,
      data: post
    });
  } catch (error) {
    res.status(400).json({
      success: false,
      message: error.message
    });
  }
}
```

 Copy

```
};

// Get all posts with advanced filtering
const getPosts = async (req, res) => {
  try {
    const page = parseInt(req.query.page) || 1;
    const limit = parseInt(req.query.limit) || 10;
    const search = req.query.search || '';
    const tag = req.query.tag || '';
    const author = req.query.author || '';
    const status = req.query.status || 'published';

    // Build filter
    let filter = { status };

    if (search) {
      filter.$or = [
        { title: { $regex: search, $options: 'i' } },
        { content: { $regex: search, $options: 'i' } }
      ];
    }

    if (tag) {
      filter.tags = { $in: [tag] };
    }

    if (author) {
      filter.author = author;
    }

    const skip = (page - 1) * limit;

    const posts = await Post.find(filter)
      .populate('author', 'name email avatar')
      .populate('comments.user', 'name avatar')
      .sort({ createdAt: -1 })
      .skip(skip)
      .limit(limit);

    const total = await Post.countDocuments(filter);

    res.status(200).json({
      success: true,
      data: posts,
      pagination: {
        page,
        limit,
        total,
        pages: Math.ceil(total / limit)
      }
    });
  } catch (error) {
    res.status(500).json({ success: false, message: error.message });
  }
};
```

```
    }  
  });  
} catch (error) {  
  res.status(500).json({  
    success: false,  
    message: error.message  
  });  
}  
};  
  
// Like a post  
const likePost = async (req, res) => {  
  try {  
    const post = await Post.findById(req.params.id);  
  
    if (!post) {  
      return res.status(404).json({  
        success: false,  
        message: 'Post not found'  
      });  
    }  
  
    // Check if user already liked the post  
    const alreadyLiked = post.likes.find(like =>  
      like.user.toString() === req.user.id  
    );  
  
    if (alreadyLiked) {  
      // Unlike the post  
      post.likes = post.likes.filter(like =>  
        like.user.toString() !== req.user.id  
      );  
    } else {  
      // Like the post  
      post.likes.push({ user: req.user.id });  
    }  
  
    await post.save();  
  
    res.status(200).json({  
      success: true,  
      data: post,  
      message: alreadyLiked ? 'Post unliked' : 'Post liked'  
    });  
  } catch (error) {  
    res.status(500).json({  
      success: false,  
      message: error.message  
    });  
  }  
};
```

```
    }  
  };  
  
  // Add comment to post  
  const addComment = async (req, res) => {  
    try {  
      const post = await Post.findById(req.params.id);  
  
      if (!post) {  
        return res.status(404).json({  
          success: false,  
          message: 'Post not found'  
        });  
      }  
  
      const comment = {  
        user: req.user.id,  
        text: req.body.text  
      };  
  
      post.comments.push(comment);  
      await post.save();  
  
      // Populate the new comment  
      await post.populate('comments.user', 'name avatar');  
  
      res.status(201).json({  
        success: true,  
        data: post.comments[post.comments.length - 1]  
      });  
    } catch (error) {  
      res.status(400).json({  
        success: false,  
        message: error.message  
      });  
    }  
  };  
  
  module.exports = {  
    createPost,  
    getPosts,  
    likePost,  
    addComment  
  };  
};
```

Advanced Data Validation

1. Custom Validation with Joi

```
// validators/post.validator.js
const Joi = require('joi');

const createPostSchema = Joi.object({
  title: Joi.string()
    .min(3)
    .max(100)
    .required()
    .messages({
      'string.min': 'Title must be at least 3 characters long',
      'string.max': 'Title cannot exceed 100 characters',
      'any.required': 'Title is required'
    }),

  content: Joi.string()
    .min(10)
    .max(2000)
    .required()
    .messages({
      'string.min': 'Content must be at least 10 characters long',
      'string.max': 'Content cannot exceed 2000 characters'
    }),

  tags: Joi.array()
    .items(Joi.string().lowercase().trim())
    .max(5)
    .messages({
      'array.max': 'You can add maximum 5 tags'
    }),

  status: Joi.string()
    .valid('draft', 'published')
    .default('draft')
});

const updatePostSchema = Joi.object({
  title: Joi.string().min(3).max(100),
  content: Joi.string().min(10).max(2000),
  tags: Joi.array().items(Joi.string().lowercase().trim()).max(5),
  status: Joi.string().valid('draft', 'published', 'archived')
});

const commentSchema = Joi.object({
  text: Joi.string()
```

 Copy

```
.min(1)
.max(500)
.required()
.messages({
  'string.min': 'Comment cannot be empty',
  'string.max': 'Comment cannot exceed 500 characters'
})
});

module.exports = {
  createPostSchema,
  updatePostSchema,
  commentSchema
};
```

2. Validation Middleware

```
// middleware/validation.middleware.js
const validate = (schema) => {
  return (req, res, next) => {
    const { error, value } = schema.validate(req.body, {
      abortEarly: false,      // Return all errors, not just first
      stripUnknown: true      // Remove unknown fields
    });

    if (error) {
      const errors = error.details.map(detail => ({
        field: detail.path.join('.'),
        message: detail.message
      }));

      return res.status(400).json({
        success: false,
        message: 'Validation error',
        errors
      });
    }

    req.body = value; // Use validated and sanitized data
    next();
  };
};

module.exports = { validate };
```

 Copy

File Upload System

1. Setup File Upload Middleware

```
npm install multer cloudinary multer-storage-cloudinary
```

[Copy](#)

```
// middleware/upload.middleware.js
const multer = require('multer');
const { CloudinaryStorage } = require('multer-storage-cloudinary');
const cloudinary = require('../config/cloudinary');

// Cloudinary storage configuration
const storage = new CloudinaryStorage({
  cloudinary: cloudinary,
  params: {
    folder: 'myapp/posts',
    allowed_formats: ['jpg', 'jpeg', 'png', 'gif', 'webp'],
    transformation: [
      { width: 800, height: 600, crop: 'limit' },
      { quality: 'auto:good' }
    ]
  }
});

// File filter
const fileFilter = (req, file, cb) => {
  // Check file type
  if (file.mimetype.startsWith('image/')) {
    cb(null, true);
  } else {
    cb(new Error('Only image files are allowed'), false);
  }
};

// Create multer upload instance
const upload = multer({
  storage: storage,
  limits: {
    fileSize: 5 * 1024 * 1024 // 5MB limit
  },
  fileFilter: fileFilter
});

// Error handling middleware for multer
const handleUploadError = (err, req, res, next) => {
  if (err instanceof multer.MulterError) {
```

[Copy](#)

```
if (err.code === 'LIMIT_FILE_SIZE') {
  return res.status(400).json({
    success: false,
    message: 'File too large. Maximum size is 5MB'
  });
}

if (err.message === 'Only image files are allowed') {
  return res.status(400).json({
    success: false,
    message: 'Only image files are allowed'
  });
}

next(err);
};

module.exports = {
  upload,
  handleUploadError
};
```

2. Cloudinary Configuration

```
// config/cloudinary.js
const cloudinary = require('cloudinary').v2;

cloudinary.config({
  cloud_name: process.env.CLOUDINARY_CLOUD_NAME,
  api_key: process.env.CLOUDINARY_API_KEY,
  api_secret: process.env.CLOUDINARY_API_SECRET
});

module.exports = cloudinary;
```

 Copy

3. File Upload Route

```
// routes/upload.routes.js
const express = require('express');
const { upload, handleUploadError } = require('../middleware/upload.middleware');
const { protect } = require('../middleware/auth.middleware');

const router = express.Router();
```

 Copy

```
// Upload single image
router.post('/image',
  protect,
  upload.single('image'),
  handleUploadError,
  async (req, res) => {
    try {
      if (!req.file) {
        return res.status(400).json({
          success: false,
          message: 'No file uploaded'
        });
      }

      res.status(200).json({
        success: true,
        message: 'File uploaded successfully',
        data: {
          url: req.file.path,
          publicId: req.file.filename
        }
      });
    } catch (error) {
      res.status(500).json({
        success: false,
        message: error.message
      });
    }
  }
);

// Upload multiple images
router.post('/images',
  protect,
  upload.array('images', 5), // Max 5 files
  handleUploadError,
  async (req, res) => {
    try {
      if (!req.files || req.files.length === 0) {
        return res.status(400).json({
          success: false,
          message: 'No files uploaded'
        });
      }

      const uploadedFiles = req.files.map(file => ({
        url: file.path,
        publicId: file.filename
      }));
```

```
    res.status(200).json({
      success: true,
      message: 'Files uploaded successfully',
      data: uploadedFiles
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: error.message
    });
  }
}
);

module.exports = router;
```

Email Integration

1. Email Service Setup

```
npm install nodemailer
```

[Copy](#)

```
// services/email.service.js
const nodemailer = require('nodemailer');

class EmailService {
  constructor() {
    this.transporter = nodemailer.createTransporter({
      host: process.env.SMTP_HOST,
      port: process.env.SMTP_PORT,
      secure: false, // true for 465, false for other ports
      auth: {
        user: process.env.SMTP_USER,
        pass: process.env.SMTP_PASS
      }
    });
  }
}

// Send welcome email
async sendWelcomeEmail(user) {
  const mailOptions = {
    from: `MyApp <${process.env.SMTP_USER}>`,
```

[Copy](#)

```

    to: user.email,
    subject: 'Welcome to MyApp!',
    html: `
      <div style="font-family: Arial, sans-serif; max-width: 600px; margin: 0
auto;">
        <h2 style="color: #333;">Welcome to MyApp, ${user.name}!</h2>
        <p>Thank you for joining our community. We're excited to have you on
board!</p>
        <p>You can now:</p>
        <ul>
          <li>Create and share posts</li>
          <li>Connect with other users</li>
          <li>Customize your profile</li>
        </ul>
        <p>If you have any questions, feel free to reach out to our support
team.</p>
        <p>Best regards,<br>The MyApp Team</p>
      </div>
    `;
  };

  try {
    await this.transporter.sendMail(mailOptions);
    console.log(`Welcome email sent to ${user.email}`);
  } catch (error) {
    console.error('Failed to send welcome email:', error);
  }
}

// Send password reset email
async sendPasswordResetEmail(user, resetToken) {
  const resetUrl = `${process.env.CLIENT_URL}/reset-password/${resetToken}`;

  const mailOptions = {
    from: `"MyApp" <${process.env.SMTP_USER}>`,
    to: user.email,
    subject: 'Password Reset Request',
    html: `
      <div style="font-family: Arial, sans-serif; max-width: 600px; margin: 0
auto;">
        <h2 style="color: #333;">Password Reset Request</h2>
        <p>Hi ${user.name},</p>
        <p>You requested a password reset. Click the button below to reset your
password:</p>
        <div style="text-align: center; margin: 30px 0;">
          <a href="${resetUrl}"
            style="background-color: #007bff; color: white; padding: 12px 24px;
text-decoration: none; border-radius: 5px; display: inline-
block;">

```

```

        Reset Password
      </a>
    </div>
    <p>This link will expire in 10 minutes.</p>
    <p>If you didn't request this, please ignore this email.</p>
    <p>Best regards,<br>The MyApp Team</p>
  </div>
  ,
};

try {
  await this.transporter.sendMail(mailOptions);
  console.log(`Password reset email sent to ${user.email}`);
} catch (error) {
  console.error('Failed to send password reset email:', error);
}
}

// Send notification email
async sendNotificationEmail(user, subject, message) {
  const mailOptions = {
    from: `MyApp <${process.env.SMTP_USER}>`,
    to: user.email,
    subject: subject,
    html: `
      <div style="font-family: Arial, sans-serif; max-width: 600px; margin: 0
auto;">
        <h2 style="color: #333;">Notification</h2>
        <p>Hi ${user.name},</p>
        <p>${message}</p>
        <p>Best regards,<br>The MyApp Team</p>
      </div>
    `
  };

};

try {
  await this.transporter.sendMail(mailOptions);
  console.log(`Notification email sent to ${user.email}`);
} catch (error) {
  console.error('Failed to send notification email:', error);
}
}

module.exports = new EmailService();

```

2. Updated Registration with Email



```
// Update auth.controller.js
const emailService = require('../services/email.service');

const register = async (req, res) => {
  try {
    const { name, email, password } = req.body;

    // Create user
    const user = await User.create({
      name,
      email,
      password
    });

    // Send welcome email (don't wait for it)
    emailService.sendWelcomeEmail(user).catch(err =>
      console.error('Welcome email failed:', err)
    );

    // Create token
    const token = user.getSignedJwtToken();

    res.status(201).json({
      success: true,
      token,
      data: { user }
    });
  } catch (error) {
    res.status(400).json({
      success: false,
      message: error.message
    });
  }
};
```



Advanced Authentication Features

1. Password Reset Flow



```
// Add to User model
const crypto = require('crypto');

userSchema.add({
  resetPasswordToken: String,
```

```
    resetPasswordExpire: Date
  });

  // Generate password reset token
  userSchema.methods.getResetPasswordToken = function() {
    // Generate token
    const resetToken = crypto.randomBytes(20).toString('hex');

    // Hash token and set to resetPasswordToken field
    this.resetPasswordToken = crypto
      .createHash('sha256')
      .update(resetToken)
      .digest('hex');

    // Set expire time (10 minutes)
    this.resetPasswordExpire = Date.now() + 10 * 60 * 1000;

    return resetToken;
  };
};
```

```
// Add to auth.controller.js
const crypto = require('crypto');
const emailService = require('../services/email.service');

// Forgot password
const forgotPassword = async (req, res) => {
  try {
    const user = await User.findOne({ email: req.body.email });

    if (!user) {
      return res.status(404).json({
        success: false,
        message: 'No user found with that email'
      });
    }

    // Get reset token
    const resetToken = user.getResetPasswordToken();

    await user.save({ validateBeforeSave: false });

    // Send email
    try {
      await emailService.sendPasswordResetEmail(user, resetToken);

      res.status(200).json({
        success: true,
```

 Copy

```
        message: 'Password reset email sent'
    });
} catch (error) {
    user.resetPasswordToken = undefined;
    user.resetPasswordExpire = undefined;
    await user.save({ validateBeforeSave: false });

    return res.status(500).json({
        success: false,
        message: 'Email could not be sent'
    });
}
} catch (error) {
    res.status(500).json({
        success: false,
        message: error.message
    });
}
};

// Reset password
const resetPassword = async (req, res) => {
    try {
        // Get hashed token
        const resetPasswordToken = crypto
            .createHash('sha256')
            .update(req.params.resettoken)
            .digest('hex');

        const user = await User.findOne({
            resetPasswordToken,
            resetPasswordExpire: { $gt: Date.now() }
        });

        if (!user) {
            return res.status(400).json({
                success: false,
                message: 'Invalid or expired token'
            });
        }

        // Set new password
        user.password = req.body.password;
        user.resetPasswordToken = undefined;
        user.resetPasswordExpire = undefined;
        await user.save();

        // Create token
        const token = user.getSignedJwtToken();
```

```
res.status(200).json({
  success: true,
  token,
  message: 'Password reset successful'
});
} catch (error) {
  res.status(500).json({
    success: false,
    message: error.message
  });
}
};
```

Chapter 2 Summary

You've now learned:

Database Mastery

- Complex queries with filtering, pagination, and search
- Database relationships and population
- Aggregation for statistics
- Virtual fields and middleware

Data Management

- Advanced validation with Joi
- File upload system with Cloudinary
- Image processing and storage
- Error handling for uploads

Communication

- Email service integration
- Welcome emails and notifications
- Password reset via email
- HTML email templates

Security & Authentication

- Password reset flow
- Token-based password reset
- Secure email delivery

- Advanced middleware

API Features

- Pagination and filtering
- Search functionality
- Like/unlike system
- Comment system
- File upload endpoints



What's Next in Chapter 3?

In the next chapter, we'll cover:

- **Real-time features** (WebSockets, Socket.io)
- **Caching strategies** (Redis)
- **API documentation** (Swagger)
- **Testing** (Unit & Integration tests)
- **Deployment** (Production setup)
- **Monitoring & Logging**
- **Performance optimization**

You now have a **feature-rich, production-ready backend** with advanced data management, file uploads, and email integration! 🎉

Ready for Chapter 3? Let me know when you want to continue!